

Operating system

Unit - V

14-12-22

Memory Management

- The main memory is division of memory management.
- The virtual memory is another division of memory management.
- The main purpose of a computer system is to execute programs.
- During execution, these programs, together with the data they access, must be in main memory (at least partially).

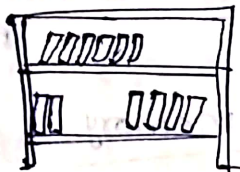
Main memory

- In CPU scheduling we showed how the CPU can be shared by a set of processes.

↓
as a result of this

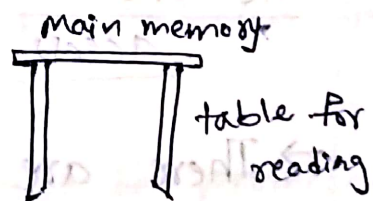
- CPU utilization, as well as speed of the computer's response to its users could be improved.

- To realize this increase in performance, however, We must keep several processes in memory i.e., We must share memory.



Book shelf

Secondary memory.



Topics to be discussed:-

→ various ways to manage memory.

→ Memory management algorithms:

- * Primitive bare-machine approach

- * Paging strategies

- * Segmentation strategies.

Basic Hardware:

→ Memory consists of a large array of words
(8) bytes, each with its own address.

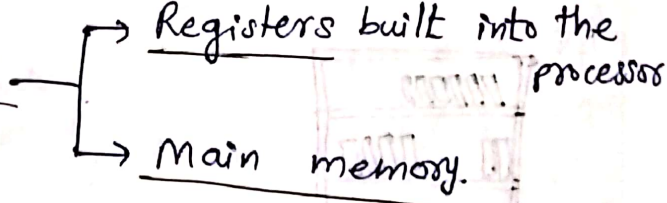
→ The CPU fetches instructions from memory
according to the value of the program counter.

→ These instructions may cause additional loading
from and storing to specific memory addresses.

- * A typical instruction-execution cycle, first fetches
an instruction from memory.

- * The instruction is then decoded and may cause
operands to be fetched from memory.

- * After the instruction has been executed on the
operands, results may be stored back in memory.

The CPU can directly  access only

→ There are machine instructions that take
memory addresses as arguments, but none
that take disk addresses i.e., secondary memory.

→ So, any instructions in execution, and any data being used by the instructions, must be in one of these direct-access storage devices.

→ If the data are not in memory, they must be moved there before the CPU can operate on them.

CPU Cycle time for accessing memories:-

Registers - Accessible within one cycle of CPU clock.

Most CPUs can decode instructions and perform simple operations ~~on~~ on registers contents at the rate of one @ more operations per clock tick.

Main memory - Access may take many cycles of the CPU clock to complete.

In this case, the processor normally needs to stall, since it does not have the data required to complete the instruction that it is executing.

This situation is intolerable because of the frequency of memory accesses.

Remedy - Add fast memory b/w the CPU and main memory. A memory buffer used to accommodate a speed differential, called a

Cache

Protection of OS from unauthorized access:-

→ The operating system has to be protected from access by user processes.

→ In addition, user processes must be protected from one another.

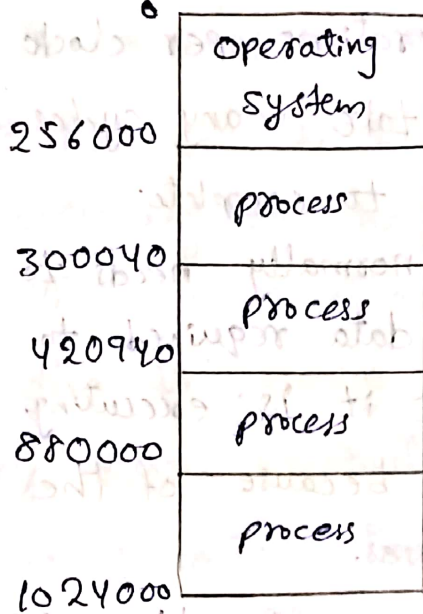
→ This protection must be provided by the hardware.

How can this be done?

Ensure that each process has a separate memory space. To do this, we need the ability to determine the range of legal addresses that the process may access and to ensure that the process can access only these legal addresses.

For this we use two registers - **BASE** and **LIMIT**.

Base and Limit

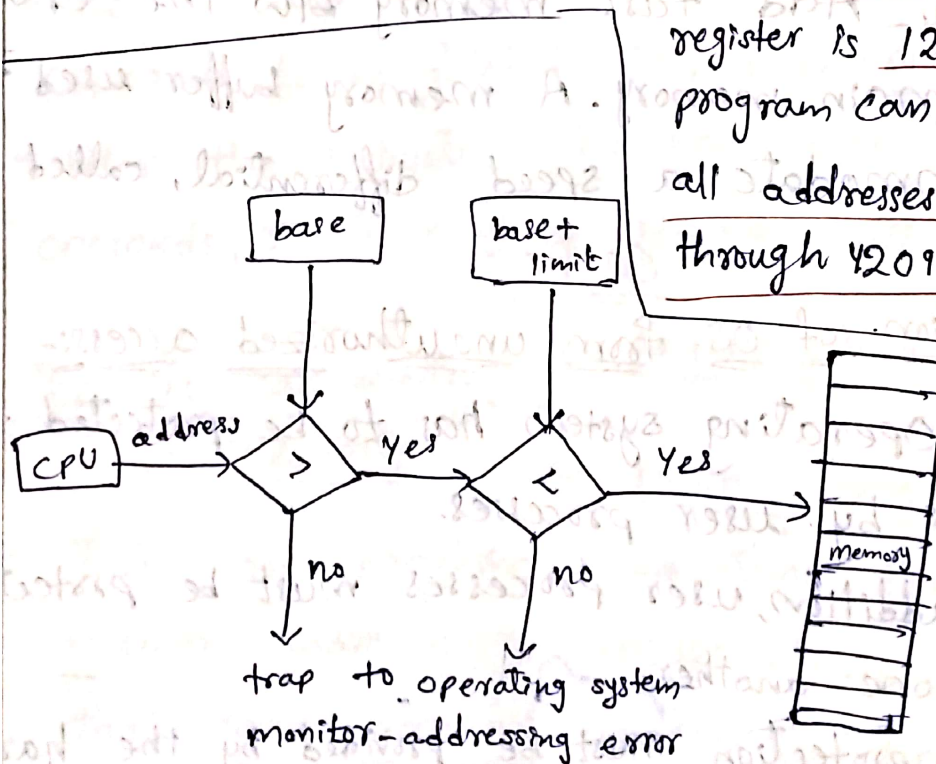


→ A base and a limit register defines a logical address space.

→ The base register holds the smallest legal physical memory address.

→ The limit register specifies the size of the range.

For ex, if the base register holds, 300040 and limit register is 120900, then the program can legally access all addresses from 300040 through 420940 (inclusive).



- The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.
- Since privileged instructions can be executed only in kernel mode, and since only the operating system executes in kernel mode, only the OS can load the base and limit registers.
- This scheme allows the OS to change the value of the registers but prevents user programs from changing the register's contents.

Address Binding

- usually, a program resides on a disk as a binary executable file.
- To be executed, the program must be brought into main memory and placed within a process.
- Depending on the memory management in use, the process may be moved b/w disk and main memory during its execution.
- The processes on the disk that are waiting to be brought into memory for execution form the *input queue*.

The sequence like this:-

Input Queue → select a process → Loads it into memory - Executes and accesses instructions and data from memory → process terminates → memory space is declared available.

- Most systems allow a user process to reside in any part of the physical memory.
- Though address space of the computer starts

at 00000 the first address of the user process need ~~to~~ not be 00000.

In most cases, a user program will go through several steps during Compile time, Load time, Execution time before being executed.

Addresses may be represented in different ways during these steps.

Source program:- Addresses are generally symbolic.
(such as count).

Compiler:- Typically binds these symbolic address to relocatable addresses (such as "14 bytes from the beginning of this module")

Linkage editor @ Loader:- Binds the relocatable address to absolute addresses (such as 74014)

Each binding is a mapping from one address space to another

Binding of instructions and data to memory addresses during COMPILE TIME

If we know at compile time where the process will reside in memory, then absolute code can be generated

Ex:- If we know that a user process will reside starting at location R, then the generated compiler code will start at that location and extend up from there.

If, at some later time, the starting location changes, then it will be necessary to recompile this code.

Binding of instructions and data to memory addresses during LOAD TIME

If we not known at compile time where the process will reside in memory, then the compiler must generate relocatable code.

In this case, final binding is delayed until load time. If the starting address changes, we need only reload the user code to incorporate this changed value.

Binding of instructions and data to memory addresses during EXECUTION TIME

If the process can be ~~to~~ moved during its execution from one memory segment to another, then binding must be delayed until run time.

Special hardware must be available for this scheme to work. Most general-purpose operating systems use this method.

Logical address space VS physical Address space

Logical address - An address generated by the CPU.

physical address - An address seen by the memory-unit that is, the one loaded into the memory-address register of the memory.

→ compile-time and Load-time Address-Binding methods generate identical Logical and physical addresses.

→ Execution-time address Binding scheme results in differing Logical and physical addresses.

→ In this case, we usually refer to the Logical address as a virtual address.

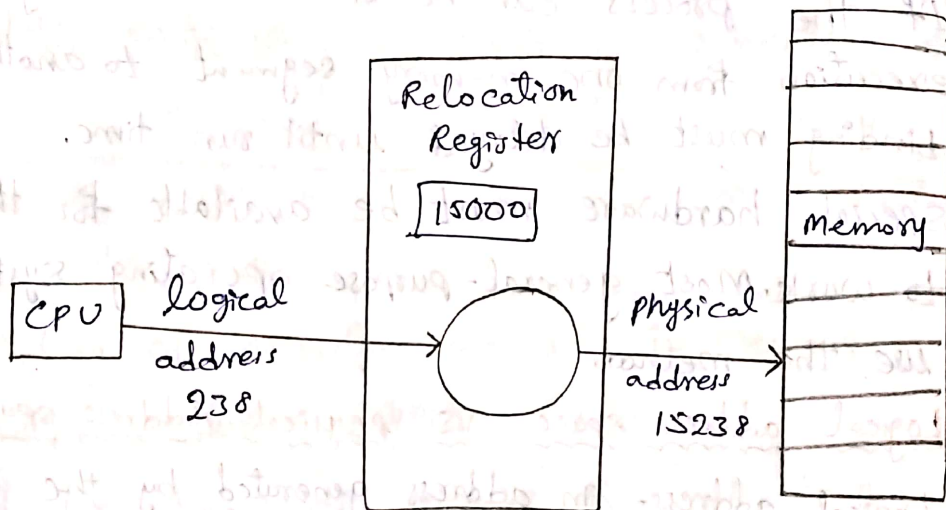
Logical Address space - The set of all logical addresses generated by a program.

Physical Address space - The set of all physical addresses corresponding to these logical addresses.

→ In the execution-time address-binding scheme, the logical and physical address spaces differ.

→ The run-time mapping from virtual to physical addresses is done by a hardware device called the **Memory-Management Unit (MMU)**.

Dynamic relocation using a relocation register → stores base address



MMU

We now have two different types of addresses, Logical addresses (in the range 0 to max) and Physical addresses (in the range $R+0$ to $R+max$ for a base value R).

→ The user generates only logical addresses and thinks that the process runs in locations 0 to max.

→ The user program supplies logical addresses; these logical addresses must be mapped to physical addresses before they are used.

Dynamic Loading :-

→ As we have discussed so far, the entire program and all data of a process must be in physical memory for the process to execute.

→ The size of a process is thus limited to the size of physical memory.

→ To obtain better memory-space utilization, we can use dynamic loading.

* With dynamic loading, a routine ^{part of program} is not loaded until it is called.

* All routines are kept on disk in a relocatable load format.

* The ^{main} main program is loaded into memory and is executed.

* When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded.

* If not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change.

* Then control is passed to the newly loaded routine.

Advantages of dynamic Loading:-

→ An unused routine is never loaded.

→ This method is particularly useful when large amounts of code are needed to handle infrequently occurring cases.

→ Although the total program size may be large, the portion that is used (and hence loaded) may be much smaller.

→ Dynamic loading does not require special support from the operating system.

→ It is the responsibility of the users to design their programs to take advantage of such a method.

Dynamic Linking and shared Libraries:

When a program runs, apart from its own modules, it also needs to use certain system Libraries as well.

Static Linking: system language libraries are treated like any other object module and are combined by the loader into the binary program image.

Disadvantage of static Linking: Each program on a system must include a copy of its language library (at least the routines referenced by the program) in the executable image. The requirement wastes both disk space and main memory.

Dynamic Linking: Checks to see whether the needed routine is already in memory. If not, the routine is loaded into memory.

→ The concept of dynamic linking is similar to that of dynamic loading. But here, linking, rather than loading, is postponed until execution time.

Dynamically Linked Libraries: DLL Files

→ DLL is a windows way of implementing this dynamic linking.

→ With dynamic linking, a stub is included in the image for each library routine reference.

Stub: A small piece of code that indicates how to locate the appropriate memory-resident library

routine (8) how to load the library if the routine is not already present.

- When the stub is executed, it checks to see whether the needed routine is already in memory.
- If not, the program loads the routine into memory.
- Either way, the stub replaces itself with the address of the routine and executes the routine.
- Thus, the next time that particular code segment is reached, the library routine is executed directly, incurring no cost for dynamic linking.

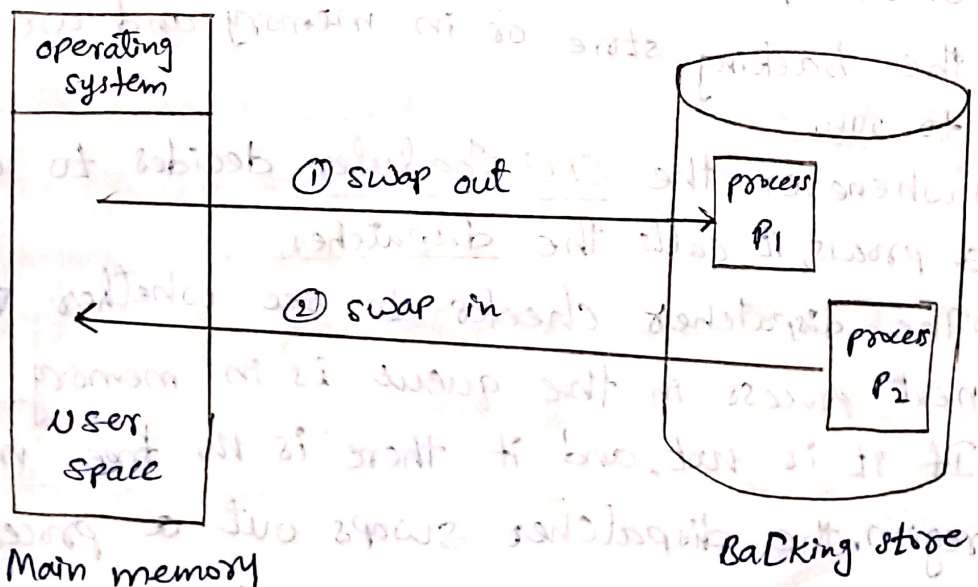
Swapping:-

- A process must be in memory to be executed.
- A process, however, can be swapped temporarily out of memory to a backing state store and then brought back into memory for continued execution.

Ex:- In a multiprogramming environment with a round-robin CPU-scheduling algorithm:

When a quantum expires, the memory manager will start to swap out the process that just finished and to swap another process into the memory spaces that has been freed.

Swapping of two processes using a disk as a backing store



Memory space Where swapping happens:-

Normally:-

A process that is swapped out will be swapped ^{back} into the same memory space it occupied previously.

→ But it depends on the address binding method.

→ If binding is done at assembly @ Load time:

The process cannot be easily moved to a different location.

→ If binding is done at execution time: The processes can be swapped into a different memory space, because the physical addresses are computed during execution time.

Backing store:-

→ swapping requires a backing state

→ It must be large enough to accommodate copies of all memory images for all users. ^{commonly a fast disk}

→ It must provide direct access to these memory images.

→ The system maintains a ready queue consisting of all processes whose memory images are on the backing store or in memory and are ready to run.

→ whenever the CPU scheduler decides to execute a process, it calls the dispatcher.

→ The dispatcher checks to see whether the next process in the queue is in memory.

→ If it is not, and if there is no free memory region, the dispatcher swaps out a process

currently in memory and swaps in the desired process.

→ It then reloads registers and transfers control to the selected process.

Swap time:-

The context-switch time in a swapping system is fairly high.

Ex:- Let user Process Size = 10MB

Transfer Rate of Backing store = 40MB per second

Actual transfer of the 10MB process to & from

main memory takes - $10000 \text{ KB} / 40000 \text{ KB per second}$

$$= \frac{1}{4} \text{ second}$$

$$= 250 \text{ milliseconds}$$

Assuming that no head seeks are necessary, and assuming an average latency of 8 milliseconds, the

$$\text{swap time} = 258 \text{ milliseconds.}$$

since we must both swap out and swap in, the

$$\text{total swap time is} = 258 \times 2$$

$$\approx 516 \text{ milliseconds}$$

→ For efficient CPU utilization, we want the execution time for each process to be long relative to the swap time.

→ Thus, in a round-robin CPU-scheduling algorithm,

for example, the time quantum should be substantially larger than 0.516 seconds

→ The major part of the swap time is transfer time

→ The total transfer time is directly proportional to the amount of memory swapped.

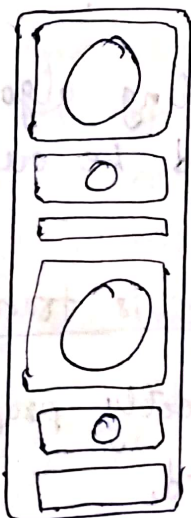
other factors that affect swapping:-

- A process must be completely idle in order to be swapped.
- A process may be waiting for an I/O operation. When we want to swap that process to free up memory,
- If the I/O is asynchronously accessing the user memory for I/O buffers, then the process cannot be swapped.

Memory Allocation:-

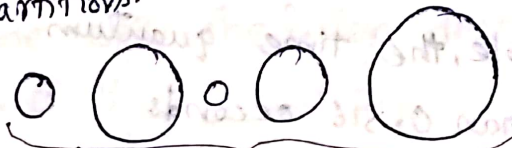
How to allocate memory?

- Divide memory into several fixed-sized partitions.
 - Each partition may contain exactly one process.
 - When a partition is free, a process is selected from the input queue and is loaded into the free partition.
 - When the process terminates, the partition becomes available for another process.
- ⇒ This is one of the simplest methods for allocating memory.



← main memory

← memory divided into several fixed-sized partitions.



processes waiting in ready queue.

→ We merge @ split the partitions on requirement then place the process in partition.

Dynamic Storage Allocation Problem:

How to satisfy a request of size n from a list of free partitions/holes in main memory?

Solutions:-

① First fit

② Best fit

③ Worst fit

First fit:- Allocate the first hole that is big enough.

→ searching can start either at the beginning of the set of holes or where the previous first-fit search ended.

→ we can stop searching - as soon as we find a free hole that is large enough.

Best fit:- Allocate the smallest hole that is big enough

→ we must search the entire list, unless the list is ordered by size.

→ This strategy produces the smallest leftover hole.

Worst fit:- Allocate the largest hole.

→ we must search the entire list, unless the list is sorted by size.

→ This strategy produces the largest leftover hole.

Fragmentation:-

As processes are loaded and removed from memory, the free memory space is broken into little pieces which results in fragmentation.

Types of Fragmentation:

1. External fragmentation
2. Internal fragmentation.

External Fragmentation:-

→ It exists when there is enough total memory space to satisfy a request, but the available spaces are not contiguous.

→ Storage is fragmented into a large number of small holes.

→ This fragmentation problem can be severe, in the worst ~~case~~ case, we could have a block of free @ wasted) memory ~~cases~~ b/w every two processes.

→ If all these small pieces of memory were in one big ~~big~~ free block instead, we might be able to run several more processes.

Internal Fragmentation:-

→ It occurs when memory blocks assigned to processes are bigger than what the process actually needs.

→ Some portion of memory is left unused, as it cannot be used by another process.

Solution:- Make use of Best-fit approach for allocating memory segments to processes.

Solution for External Fragmentation:- Compaction

Shuffle the memory contents so as to place all free memory together in one large block.

problems with compaction:-

→ compaction is not always possible.

→ If relocation is static and is done at assembly @ load time, compaction cannot be done.

→ It is possible only if relocation is dynamic and is done at execution time.

Paging:-

→ Paging is a memory-management scheme that permits the physical address space of a process to be non-contiguous.

→ Paging avoids the considerable problem of fitting memory chunks of varying sizes onto the backing store.

→ Most memory-management schemes that we discussed before suffered from this problem.

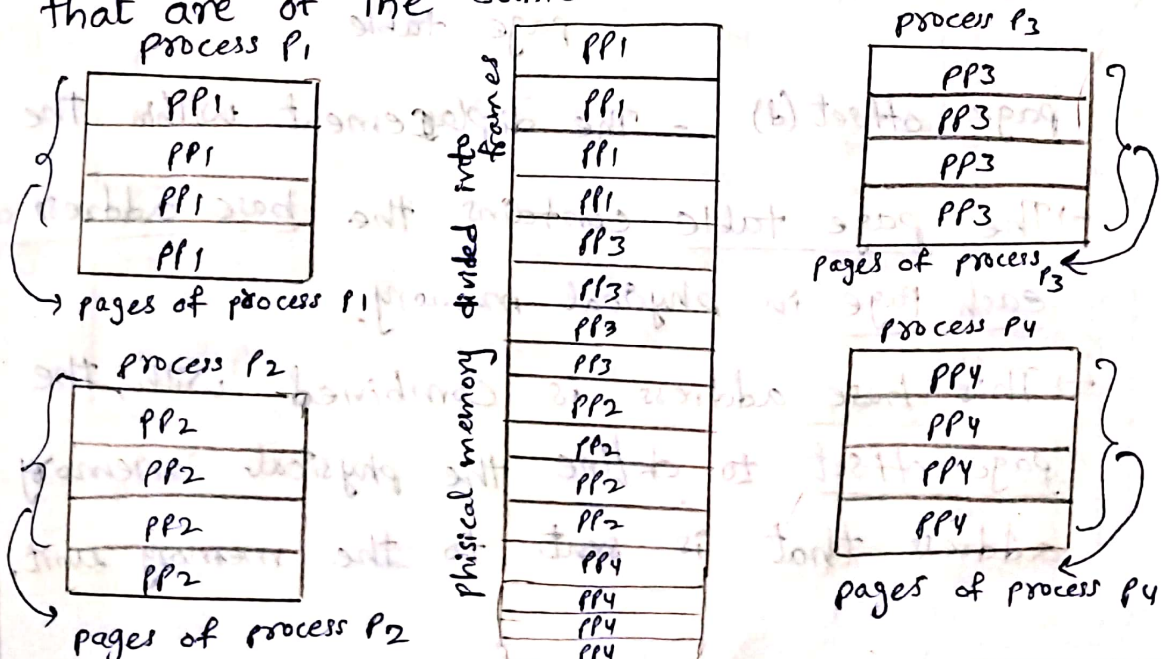
Basic method of paging:-

→ Break physical memory into fixed-sized blocks called frames

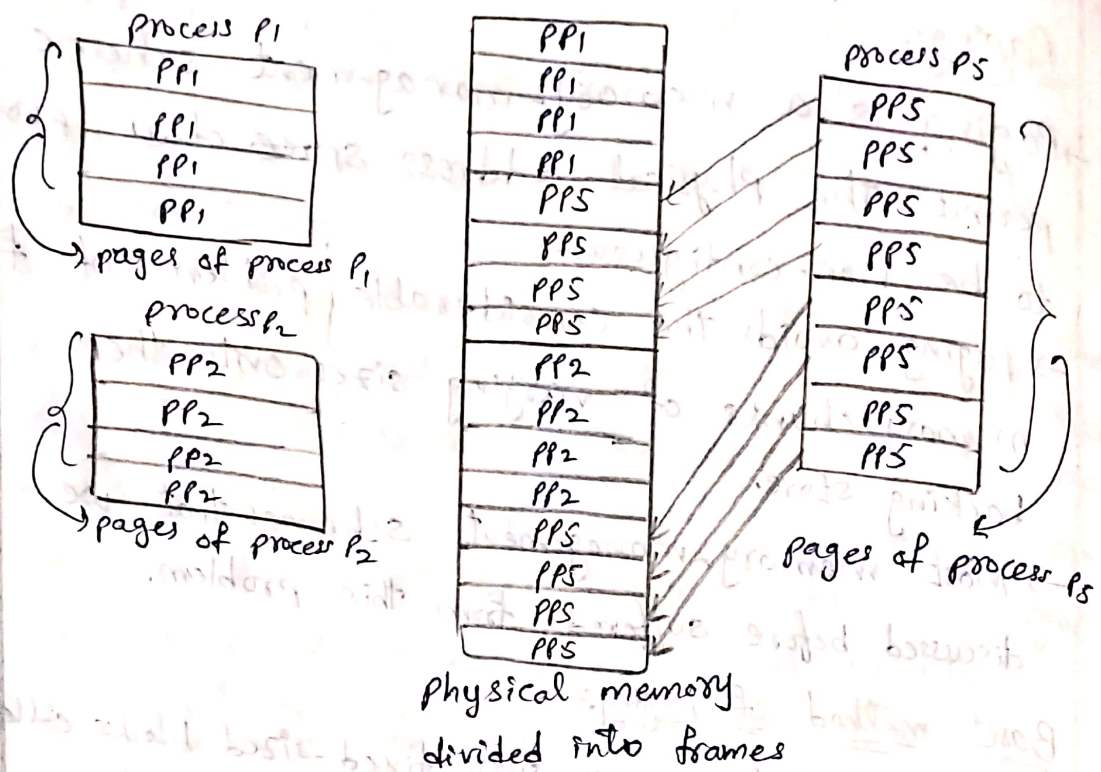
→ Breaks logical memory into blocks of the same size called pages.

→ When a process is to be executed, its pages are loaded into any available memory frames from the backing store.

→ The backing store is divided into fixed-sized blocks that are of the same size as the memory frames.



If P_3 and P_4 completes its execution then process P_5 arrived with 8 pages then all the 8 pages of P_5 are allocated in the free frames.



→ Paging solves the problem of external fragmentation.

Page Table:-

→ Every address generated by the CPU is divided into two parts:

A page number (p) and A page offset (d)

Page Number (p) - Used as an index into a page table

page offset (d) - The displacement within the page

→ The page table contains the base address of each page in physical memory.

→ This base address is combined with the page offset to define the physical memory address that is sent to the memory unit.

paging model of logical and physical memory

page 0
page 1
page 2
page 3

Logical Memory

pagenum	frame numbers
0	1
1	4
2	3
3	7

page table

0	
1	Page 0
2	
3	Page 2
4	Page 1
5	
6	
7	Page 3

Translation of Logical Address to physical address using Page Table:-

Every address generated by the CPU is divided into two parts: A page number (P) & A page offset (d)

page number	page offset
P	d

page 0	0 1 2 3	a b c d
page 1	4 5 6 7	e f g h
page 2	8 9 10 11	i j k l
page 3	12 13 14 15	m n o p

Logical memory

0	5
1	6
2	7
3	2

page table

Frame 0

Frame 1

Frame 2

Frame 3

Frame 4

Frame 5

Frame 6

Frame 7

0		
1		
2		
3		
4	i	
5	j	
6	k	
7	l	
8	m	
9	n	
10	o	
11	p	
12		
13		
14		
15		
16		
17		
18		
19		
20	a	
21	b	
22	c	
23	d	
24	e	
25	f	
26	g	
27	h	
28		
29		
30		
31		

physical memory



① Logical Address 0 maps to physical Address 20

$$\text{Formula: } [(\text{Frame} \times \text{Page size}) + \text{offset}]$$

$$((5 \times 4) + 0) = 20$$

② Logical Address 3 maps to physical Address 23

$$\text{Formula: } (\text{Frame} \times \text{Page size}) + \text{offset}$$

$$((5 \times 4) + 3) = 23$$

③ Logical Address 4 maps to physical Address 24

$$(\text{Frame} \times \text{Page size}) + \text{offset}$$

$$((6 \times 4) + 0) = 24$$

④ Logical Address 13 maps to physical Address 9

$$(\text{Frame} \times \text{Page size}) + \text{offset}$$

$$((2 \times 4) + 1) = 9$$

Hardware Implementation of Page Table:

Case-1: Implement the page table as a set of dedicated registers.

problem: This can be used only when page table is reasonably small.

Case-2: keep the page table in main memory and a page-table base register (PTBR) points to the page table.

problem: The time required to access a user memory location is increased as there are two memory accesses needed to access a byte (one for the page-table entry, one for the byte).

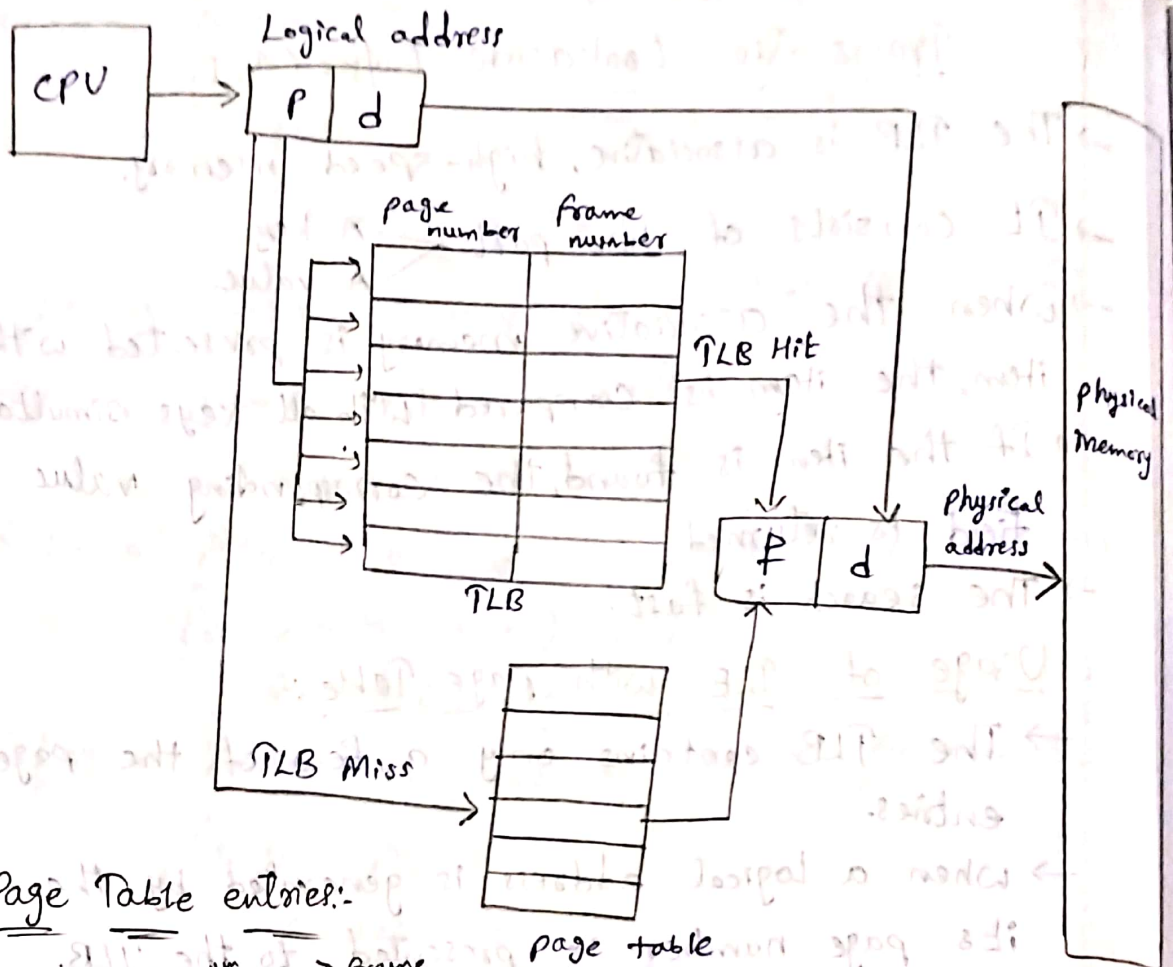
Solution: make use of

Translation Lookaside Buffer (TLB)

- The TLB is associative, high-speed memory.
- It consists of two parts $\left\{ \begin{array}{l} \text{A key} \\ \text{A value} \end{array} \right.$
- When the associative memory is presented with an item, the item is compared with all keys simultaneously.
- If the item is found, the corresponding value field is returned.
- The search is fast.

Usage of TLB with page Table:-

- The TLB contains only a few of the page-table entries.
- When a logical address is generated by the CPU, its page number is presented to the TLB.
- If the page number is found, its frame number is immediately available and is used to access memory. This is TLB Hit.
- If the page number is not in the TLB, a memory reference to the page table must be made. $\hookrightarrow$ this is TLB miss.
- When the frame number is obtained we can use it to access memory.
- Also we add the page number and frame number to the TLB, so that they will be found quickly on the next reference.
- The replacement is done for page table entries which are the least used @ randomly.



Page Table entries:-

page num	Frame number
0	1
1	4
2	3
3	7

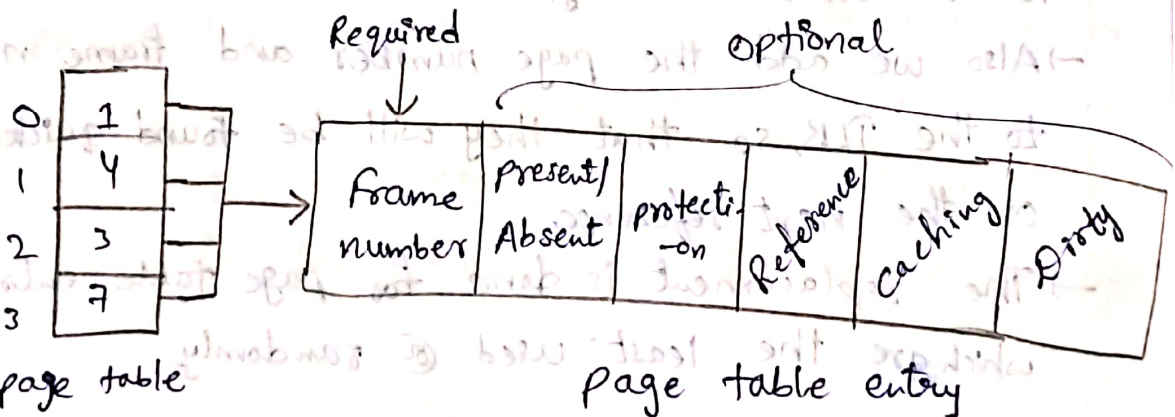
Page Table

page table

→ page table entries contains several information about pages which vary from OS to OS

→ The most important information in a page table entry is the frame number

→ The remaining informations are optional



Frame number:-

- Frame number denotes the frame where the page is present in the main memory.
- The no. of bits required for this depends on the number of frames in the main memory.

Present / Absent bit:-

- The present / Absent bit specifies whether the page is present in the main memory @ not.
- It is also called as valid / Invalid bit.
- If the page we are looking for is not present in main memory, it is called PAGE FAULT.
- If the page we are looking for it is not present in the main memory, the present / Absent bit is set to 0.

Protection bit:-

- The protection bit also known as Read / write bit is used for page protection.
- It specifies the permission for read @ write operations on the page.
- The bit is set to 0 if only read operation is allowed.
- The bit is set to 1 if both read & write operations are allowed.

Reference bit:-

- The Reference bit specifies whether the page has been referenced in the last clock cycle @ not.
- It is set to 1 when the page is accessed.
- It is set to 0 if it is not accessed during the last clock cycle.

Caching bit:-

- The caching bit is used for enabling @ disabling caching of the page.
- When we need fresh data we have to disable caching so as to avoid fetching of old data from the cache.

- When caching has to be disabled, this bit is set to 1, otherwise it is set to 0.

Dirty bit:-

- The Dirty bit is also known as the modified bit.
- It specifies whether the page has been modified @ not.
- If the page has been modified, then this bit is set to 1 otherwise set to 0.
- This bit helps in avoiding unnecessary writes to the secondary memory when a page is being replaced by another page.

Shared pages:-

- An advantage of paging is the possibility of sharing common code.
- This is particularly important in a time-sharing environment.

ex: consider a system that supports 40 users, each of whom execute a text editor.

If the text editor consists of 150KB of code and 50KB of data space, we need:

$$(150 \times 40) + (50 \times 40) \Rightarrow 6000 + 2000$$

= 8000 KB to support the 40 users.

→ However, if the code is reentrant code (or pure code), it can be shared among different processes.

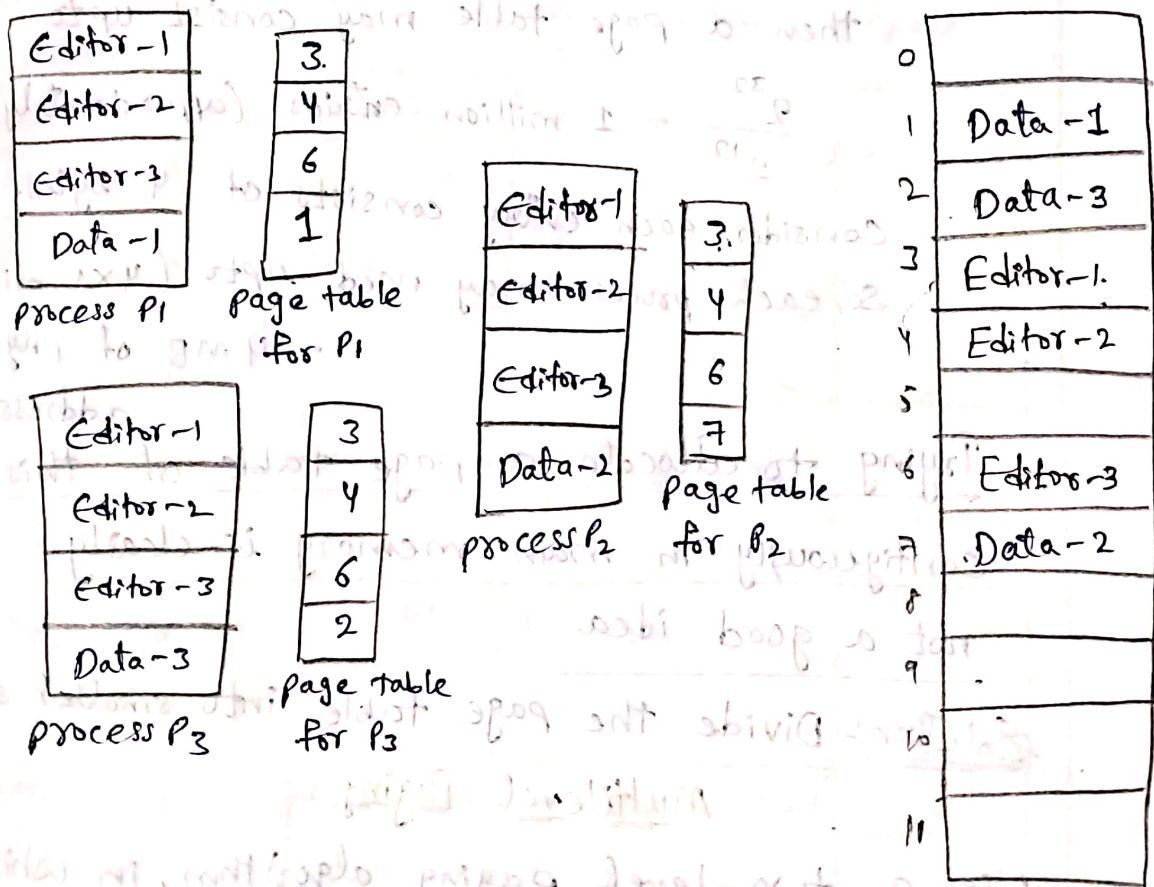
→ reentrant code:- The code that is non-self-modifying code. It never changes during execution.

→ Thus, two or more processes can execute the same code at the same time.

→ Each process has its own copy of registers and data storage to hold the data for the process's execution.

→ The data for two different processes will, of course, be different.

sharing of code in a paging environment



Here these editors are being shared by processes

we saving the memory by sharing the memory.

In previous example, if we share the memory

$$150 + (50 \times 40)$$

$$= 150 + 2000$$

= 2150 KB to support 40 users.

Hierarchical paging:-

→ Most modern computer systems support a large logical address space (2^{32} to 2^{64}).

→ In such an environment, the page table itself becomes excessively large.

Ex:- Consider a system with:

32-bit logical address space.

If page size = 4 KB i.e., 2^{12} bytes

then a page table may consist upto

$$\frac{2^{32}}{2^{12}} = 1 \text{ million entries (approximately).}$$

Consider each entry consists of 4 bytes.

So, each process may need upto (4 × 1 million) bytes
= 4 MB of physical address space.

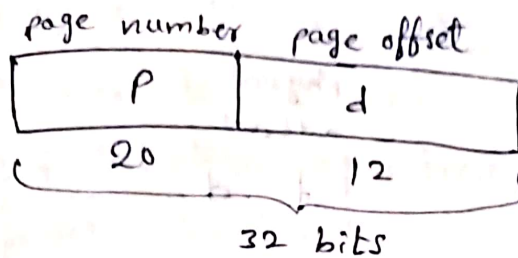
Trying to allocate a page table of this size contiguously in main memory is clearly not a good idea

Solution:- Divide the page table into smaller sizes.

Multilevel Paging

Use a two-level paging algorithm, in which the page table itself is also paged.

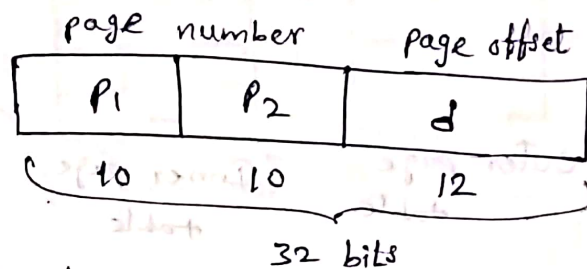
Normal paging:-



page number (P) - used as an index into a page table.

page offset (d) - The displacement within the page.

Multilevel paging:-

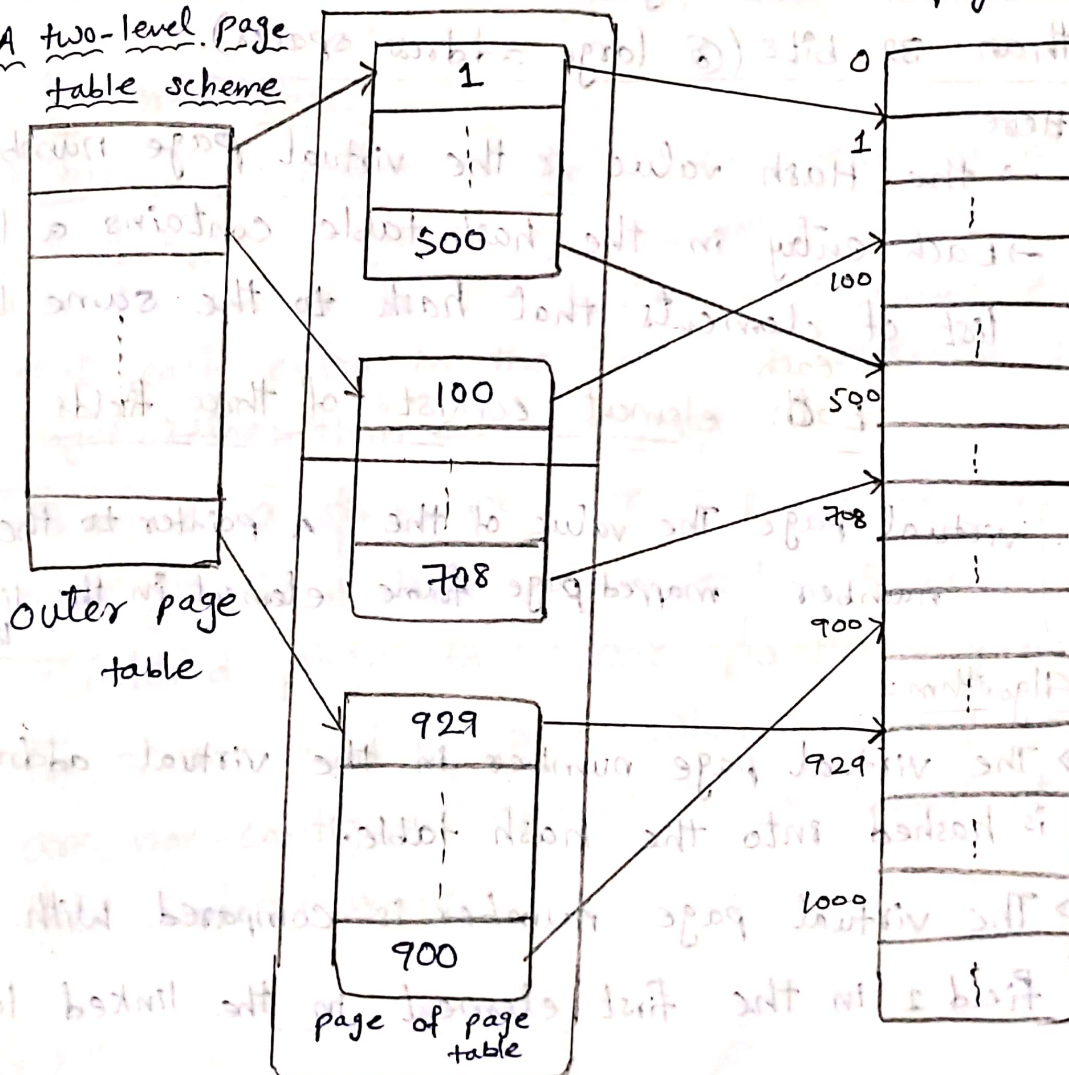


P₁ - index into the outer page table

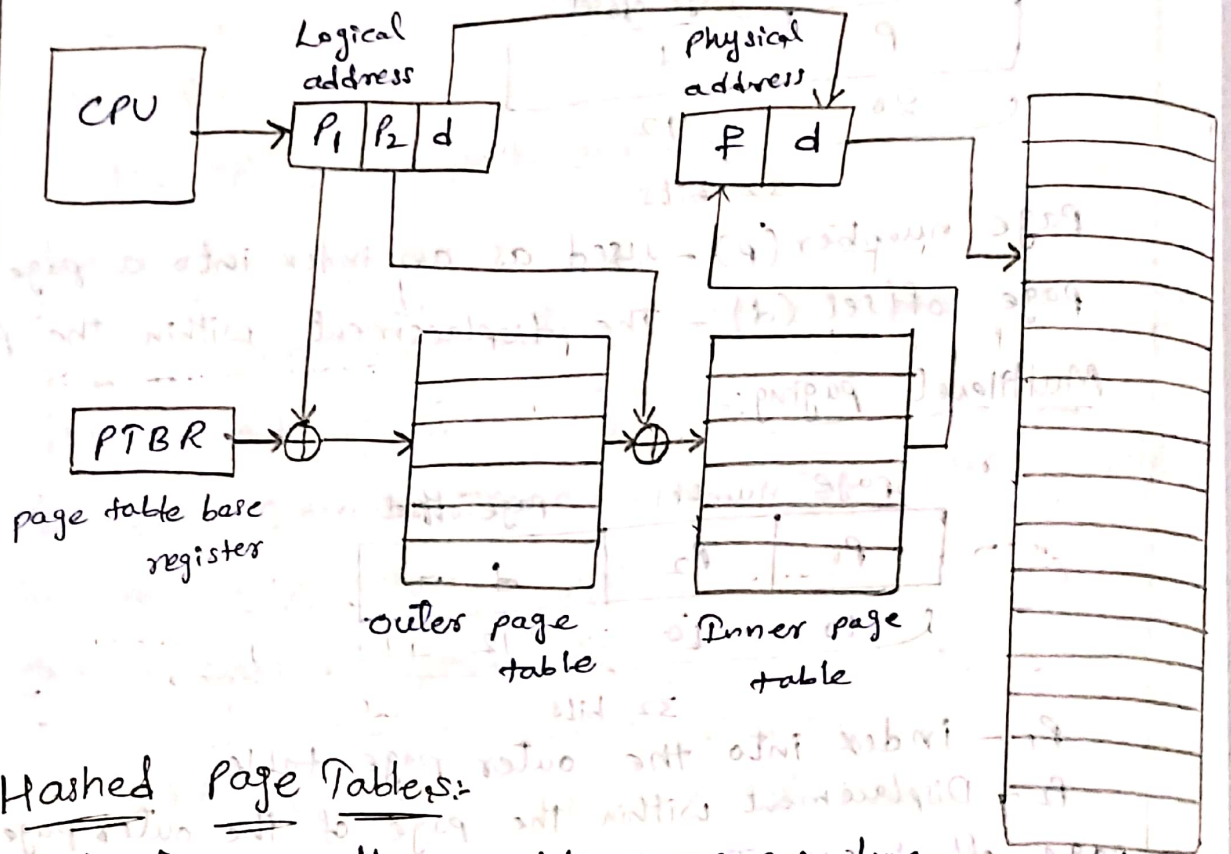
P₂ - Displacement within the page of the outer page table

page offset (d) - The displacement within the page.

A two-level page table scheme



Address translation for a two-level system



Hashed Page Tables:

Used for handling address spaces larger than 32-bits (@ large address spaces). Main memory

Here

- the Hash value is the virtual page number
- Each entry in the hash table contains a linked list of elements that hash to the same location

Each element consists of three fields

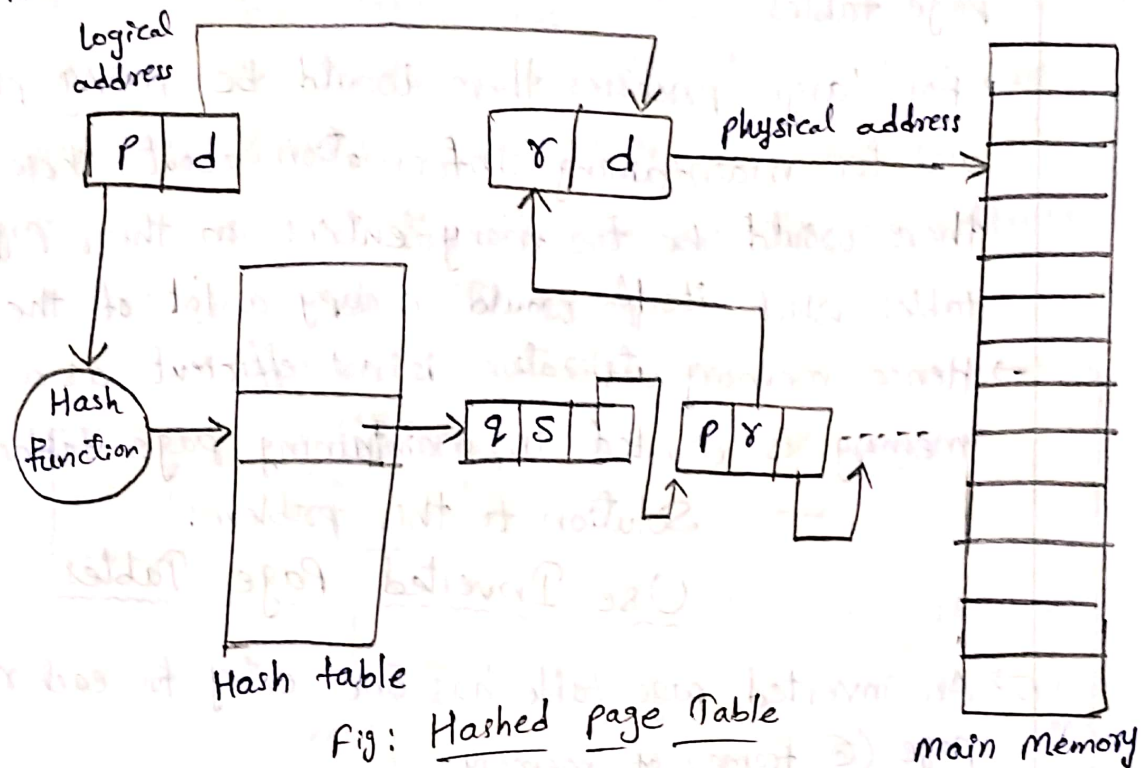
virtual page number	The value of the mapped page frame	A pointer to the next element in the linked list
---------------------	------------------------------------	--

Algorithm:-

- The virtual page number in the virtual address is hashed into the hash table.
- The virtual page number is compared with field 1 in the first element in the linked list.

→ If there is a match, the corresponding page frame (field 2) is used to form the desired physical address.

→ If there is no match, subsequent entries in the linked list are searched for matching virtual page number.



Clustered Page Tables:-

→ This is a variation of the hashed page tables which is favorable for 64-bit address.

→ It is similar to the hashed page tables except that each entry in the hash table refers to several pages rather than a single page.

→ Therefore, a single page-table entry can store the mappings for multiple physical-page frames.

→ clustered page tables are particularly useful for sparse address spaces, where memory references are non-contiguous and scattered throughout the address space.

Inverted page Tables:-

- In most operating systems, a separate page table is maintained for each processes.
- So, for 'n' no. of processes there will be 'n' no. of page tables.
- For large processes, there would be many pages and for maintaining information about these pages, there would be too many entries in their page tables which itself would occupy a lot of the memory.
- Hence memory utilization is not efficient as a lot of memory is wasted in maintaining page tables itself.

Solution to this problem:

Use Inverted Page Tables

- An inverted page table has one entry for each real page (@ frame) of memory.
- Each entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page.
- Thus, only one page table is in the system, and it has only one entry for each page of physical memory.

Each virtual address in the system consists of

process-id	page-number	offset
------------	-------------	--------

three fields

Each inverted page-table entry is a pair

process-id	page-number
------------	-------------

The Working:-

- When a memory reference occurs, part of the virtual address, consisting of $\langle \text{process-id}, \text{page-num} \rangle$, is presented to the memory subsystem.
- The inverted page table is then searched for a match.
- If a match is found—say at entry 'i', then the physical address $\langle i, \text{offset} \rangle$ is generated.
- If no match is found, then an illegal address access has been attempted.

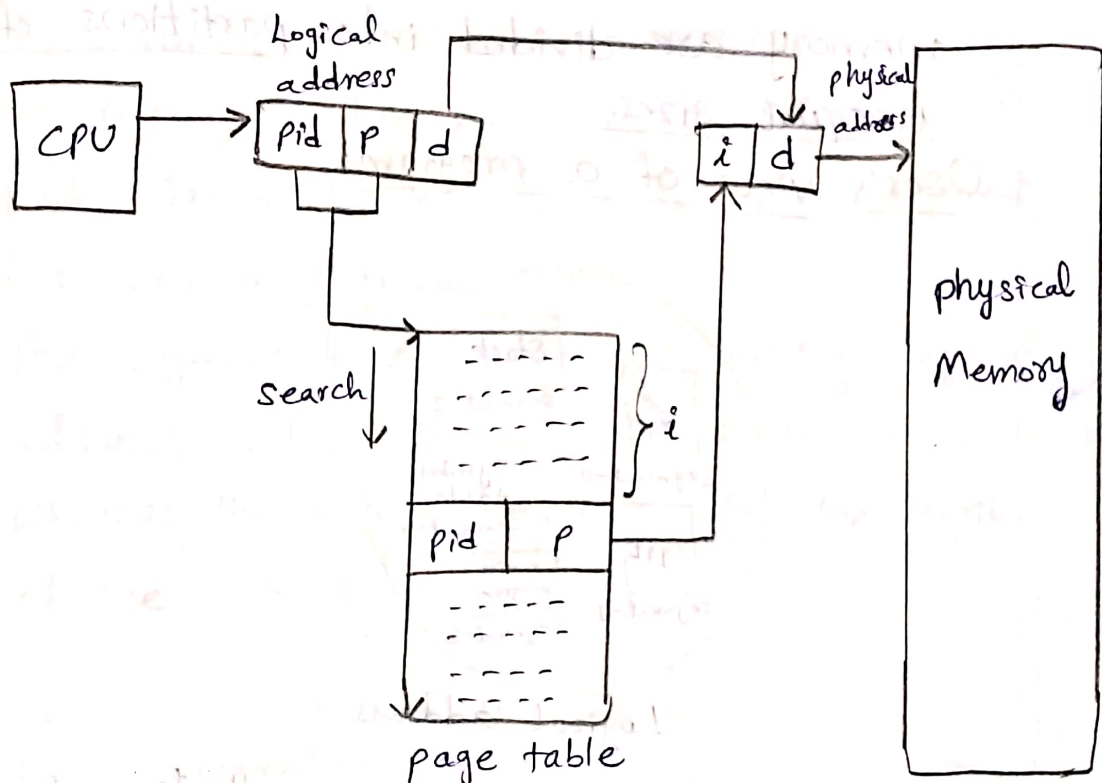


Fig: Inverted page table

Advantages of Inverted page table :-

- Reduces Memory usage

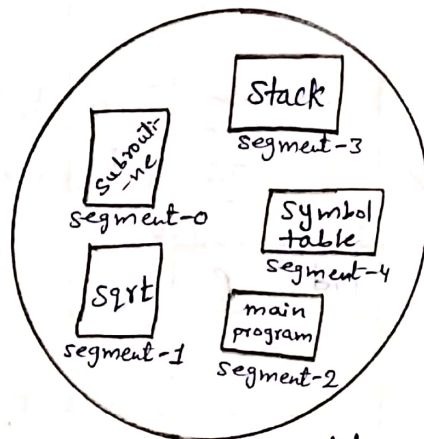
Disadvantages of Inverted Page Tables:-

- Increased search time as inverted page table is sorted by physical address, but lookups occur on virtual addresses. The whole table might need to be searched for a match.
- Difficulty in implementing shared memory.

Segmentation:-

- Segmentation is another non-contiguous memory allocation technique like paging.
- Unlike paging, in segmentation, the processes are not divided into fixed pages.
- Instead, the processes are divided into several modules called segment while improves the visualization for the users.
- So, here, both secondary memory and main memory are divided into partitions of unequal sizes.

User's view of a program:-



Logical address

- A logical address space is a collection of segments.
- Each segment has a name and a length.
- The address specify both the segment name and the offset within the segment.
- The user therefore specifies each address by two quantities: A segment name and An offset.

While in paging- the user specifies only a single address, which is partitioned by the hardware into a page number and an offset, all invisible to

the programmer.

→ For simplicity of implementation, segments are numbered and are referred to by a segment number, rather than by a segment name.

thus, a logical address consists of a two tuple

Segment-number	offset
----------------	--------

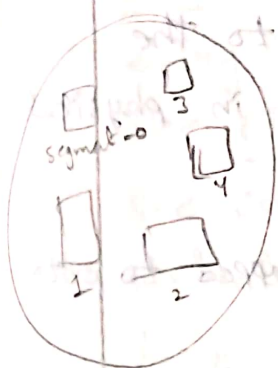
Segment table:-

→ We must define an implementation to map two-dimensional user-defined addresses into one-dimensional physical addresses.

→ This mapping is done by using a segment table.

→ Each entry in the segment table has a segment base and a segment limit.

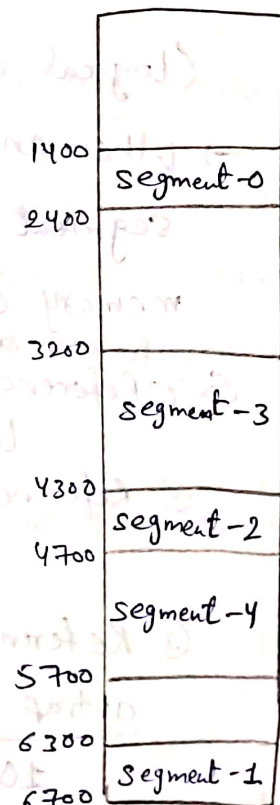
→ The segment base contains the starting physical address where the segment resides in memory, whereas the segment limit specifies the length of the segment.



logical address space

	Limit	Base
segment-0	1000	1400
segment-1	400	6300
segment-2	400	4300
segment-3	1100	3200
segment-4	1000	4700

segment table



physical memory

An example of Segmentation

Segmentation Hardware

→ Although in segmentation the user can now refer to objects in the program by a two-dimensional address, the actual physical memory is still, of course, a one-dimensional sequence of bytes.

→ Thus, we must define an implementation to map two-dimensional user-defined addresses into one-dimensional physical addresses.

→ This is done with the help of a segment table.

→ A logical address consists of two parts:

A segment num, 's' and, an offset into that segment, 'd'

→ The segment number is used as an index to the segment table.

→ The offset, 'd' of the logical address must be b/w 0 and the segment limit.

→ If it is not, we trap to the operating system

(logical addressing attempt beyond end of segment).

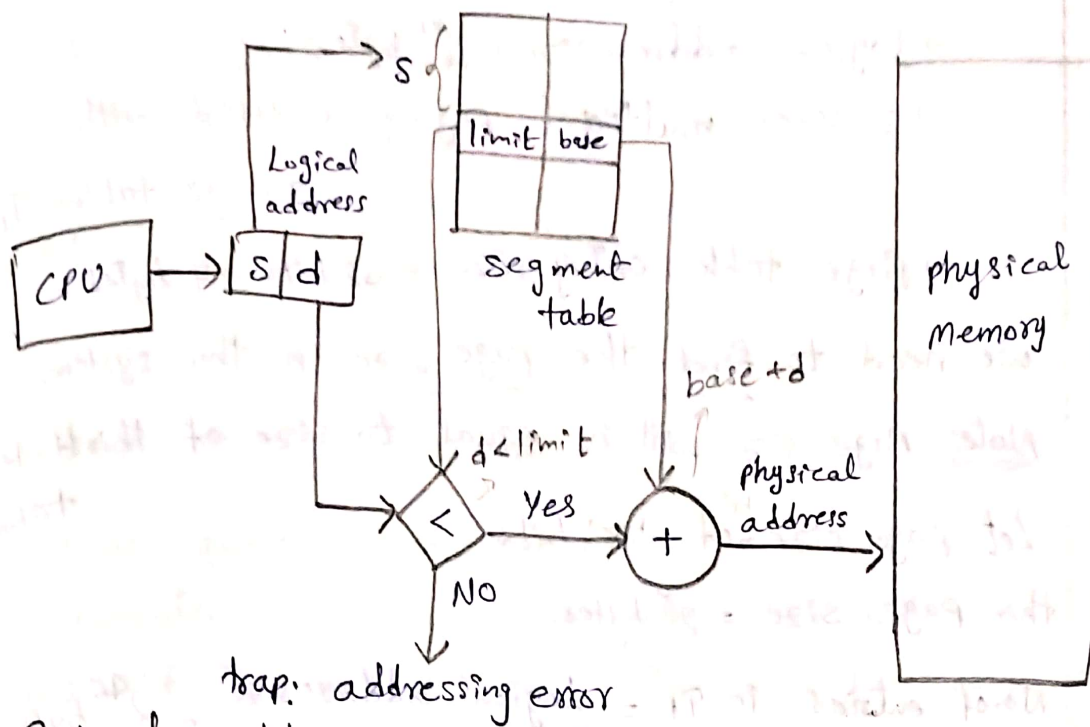
→ When an offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte.

→ For ex in previous page

Ex: ① Reference to byte 53 of segment-2-mapped to onto location $4300 + 53 = 4353$

② Reference to byte 852 of segment-3-mapped to onto location $3200 + 852 = 4052$

③ Reference to byte 1222 of segment 0 would result in a trap to the OS, as this segment is only 1000 bytes long



Solved problems:-

Q1) A computer uses 46-bit virtual address, 32 bit physical address, and a three-level paged page table organization.

The page table base register stores the base address ^{exactly} of the first-level table (T_1), which occupies one page. Each entry of T_1 stores the base address of a page of the ~~see~~ second-level table (T_2). Each entry of T_2 stores the base address of a page of the third-level table (T_3). Each entry of T_3 stores a page table entry (PTE).

The PTE is 32 bits in size. The processor used in the computer has a 1 MB 16 way set associative virtually indexed physically tagged cache. The cache block size is 64 bytes.

What is the size of a page in KB in the computer?

- (A) 2
- (B) 4
- ~~(C) 8~~
- (D) 16

Sol: Given

→ logical address size = 2^{46} bytes

→ 3 level multilevel paging is used with

3 page tables T_1, T_2, T_3

→ page table entry size = 32 bits = 4 bytes

we need to find the page size in this system

Note: Page size will be equal to size of third-level table T_3 .

Let page size be 'x' bits

then page size = 2^x bytes.

$$\text{No. of entries in } T_1 = \frac{\text{Logical address size}}{\text{page size}} = \frac{2^{46} \text{ bytes}}{2^x \text{ bytes}} = 2^{46-x}$$

$$\begin{aligned} \text{Size of } T_1 &= \text{No. of entries in page table} \times \text{PTE size} \\ &= (2^{46-x}) * (4 \text{ bytes}) \\ &= (2^{46-x}) * (2^2) \\ &= \underline{2^{48-x} \text{ bytes}} \end{aligned}$$

$$\text{No. of entries in } T_2 = \frac{\text{Size of } T_1}{\text{page size}} = \frac{2^{48-x} \text{ bytes}}{2^x \text{ bytes}} = 2^{48-2x}$$

$$\begin{aligned} \text{Size of } T_2 &= \text{No. of entries in page table} \times \text{PTE size} \\ &\Rightarrow (2^{48-2x}) * (4 \text{ bytes}) = \underline{2^{50-2x} \text{ bytes}} \end{aligned}$$

$$\text{No. of entries in } T_3 = \frac{\text{Size of } T_2}{\text{page size}} = \frac{2^{50-2x} \text{ B}}{2^x \text{ B}} = 2^{50-3x}$$

$$\begin{aligned} \text{Size of } T_3 &= \text{No. of entries in page table} \times \text{PTE size} \\ &= (2^{50-3x}) * (4 \text{ bytes}) = \underline{2^{52-3x} \text{ bytes}} \end{aligned}$$

page size will be equal to size of third-level table T_3

Size of T_3 = page size

$$2^{52-3x} = 2^x$$

$$52-3x = x$$

$$52 = 4x$$

$$x = 13 \text{ bits}$$

$$\therefore \text{page size} = 2^{13} = 8129 \text{ bytes} = \frac{8129}{2^{10}} = \frac{8129}{1024} = \underline{\underline{8 \text{ KB}}}$$

Q2) Same question as previous data.

A computer... T_3 stores a page table entry (PTE).

The PTE is 32bits in size.... The cache block size is 64 bytes.

What is the minimum no. of page colors needed to guarantee that no two synonyms map to different sets in the processor cache of this computer?

(A) 2 (B) 4 (C) 8 (D) 16

Given that PTE is 32bits in size. The processor used in the computer has a 1MB 16 way set associative virtually indexed physically tagged cache. The cache block size is 64 bytes.

Given $\rightarrow$ cache size = 1MB

$\rightarrow$ Uses 16 way set associative virtually indexed physically tagged cache

$\rightarrow$ cache block size = 64 bytes

$\rightarrow$ page size = 8KB [from previous solution]

$$\text{No. of pages in cache} = \frac{\text{Cache size}}{\text{Page size}} = \frac{1 \text{ MB}}{8 \text{ KB}} = \frac{2^{20} \text{ bytes}}{2^{13} \text{ bytes}} = 2^7 = \underline{\underline{128}}$$

$$\text{No. of sets} = \frac{\text{No. of pages in cache}}{16} = \frac{128}{16} = \underline{\underline{8}}$$

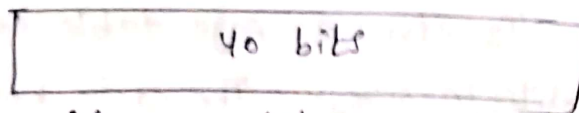
$\therefore$ Minimum no. of page colors needed to guarantee that no two synonyms map to different sets in the processor cache = 8



Q3) A computer system implements a 40 bit virtual address, page size of 8 kilobytes, and a 128-entry translation look-aside buffer (TLB) organized into 32 sets each having four ways. Assume that the TLB tag does not store any process id.

The minimum length of TLB tag in bits is 22
(A) 20 (B) 10 (C) 11 (D) 22

Sol:- virtual @ Logical address



virtual address = 40 bits

since TLB is organized into 32 sets, set offset = 5 bits

since page size = 8 KB, word offset = 13 bits

$$\because 2^5 = 32$$

$$\because 8 \text{ KB} = 8192 \text{ bytes} \\ = 2^{13}$$

Minimum Length of the TLB tag =

= Total logical address size - set offset - word offset

$$= 40 - 5 - 13$$

$$= \underline{22 \text{ bits}}$$

Q4) Consider a system with byte-addressable memory, 32 bit logical addresses, 4 kilobyte page size and page table entries of 4 bytes each.

The size of the page table in the system in megabytes is 4

(A) 2 (B) 4 (C) 8 (D) 16

Given: No. of bits in logical address = 32 bits

page size = 4 KB

page table entry size = 4 bytes.

We have to find size of the page.

calculate the process size:-

No. of bits in logical address = 32 bits

so, process size = 2^{32} bytes

= 4 GB

calculate the No. of entries in page table:-

No. of entries in page table = $\frac{\text{process size}}{\text{page size}} = \frac{4 \text{ GB}}{4 \text{ KB}}$

$$= \frac{2^{32} \text{ bytes}}{2^{12} \text{ bytes}} = 2^{20} \text{ entries}$$

calculate the page table size:-

Page table size = No. of entries in page table * page table entry size

$$= 2^{20} * 4 \text{ bytes}$$

$$= 2^{22} \text{ bytes}$$

$$= 4194304 \text{ bytes}$$

$$= 4194304$$

$$= 4 \text{ MB}$$

$$1024 \times 1024$$

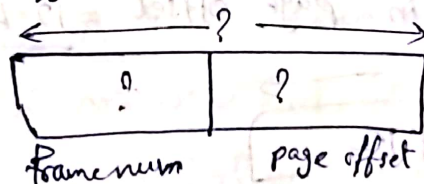
Q5) Consider a machine with 64 MB physical memory and a 32 bit virtual address space.

If the page size is 4KB, what is the approximate size of the page table?

(A) 16 MB (B) 8 MB (C) 2 MB (D) 24 MB

Approach:-

Find the no. of bits in physical address - frame number and page offset.



calculate process size and with the help of that

Calculate the no. of entries in page table.

Page table size = no. of entries in page table $\times$
Page table entry size (which is
equal to no. of bits in frame num.)

Given:-

Size of physical @ main memory = 64 MB.

No. of bits in virtual addresses space = 32 bits

Page size = 4 KB

We have to find size of the page table:-

Calculate the no. of bits in physical address:-

Size of physical @ main memory = 64 MB
 $= 2^{26}$ bytes

So, no. of bits in physical address = 26 bits

Calculate the no. of bits in frame number:-

No. of frames in main memory = $\frac{\text{Size of main memory}}{\text{Frame size}}$

$$= \frac{64 \text{ MB}}{4 \text{ KB}} = \frac{2^{26} \text{ bytes}}{2^{12} \text{ bytes}}$$

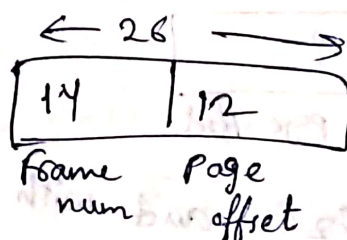
$$= 2^{14}$$

So, Number of bits in frame number = 14 bits

Calculate the no. of bits in page offset:-

Page size = 4 KB = 2^{12} bytes

So, No. of bits in page offset = 12 bits.



Calculate the process size:-

No. of bits in virtual address space = 32 bits

$$\therefore \text{process size} = 2^{32} \text{ bytes} = 4 \text{ GB}$$

Calculate the no. of entries in page table:-

$$\begin{aligned} \text{No. of entries in page table} &= \frac{\text{process size}}{\text{page size}} = \frac{4 \text{ GB}}{4 \text{ KB}} = \frac{2^{32} \text{ bytes}}{2^{12} \text{ bytes}} \\ &= 2^{20} \text{ pages} \end{aligned}$$

Calculate the size of page table:-

$$\text{size of page table} = \text{No. of entries in page table} \times \text{Page table entry size}$$

$$= \text{No. of entries in page table} \times \text{No. of bits in Frame number}$$

$$= 2^{20} \times 14 \text{ bits}$$

$$= 2^{20} \times 16 \text{ bits (approx } 14 \text{ bits} \approx 16 \text{ bits)}$$

$$= 2^{20} \times 2 \text{ bytes}$$

$$= 2^{21} \text{ bytes} = \frac{2097152}{1024 \times 1024} = 2 \text{ MB}$$

Q6) For each of the four processors P_1, P_2, P_3 and P_4 , the total size in kilobytes (KB) and the no. of segments are given below.

The page size is 1 KB, the size of an entry in the

page table is 4 bytes. The size of an entry in the

Segment table is 8 bytes. The maximum size of a

Segment is 256 KB. The paging method for memory

management uses two-level paging, and its storage

overhead is P. The storage overhead for the

segmentation method is S. The storage overhead

for the segmentation and paging method is T.

What is the relation among the overheads for

the different methods of memory management in the concurrent execution of the below four processes?

Process	Total size (in kB)	No. of segments	(A) $P < S < T$ (B) $S < P < T$ (C) $S < T < P$ (D) $T < S < P$
P ₁	195	4	
P ₂	254	5	
P ₃	45	8	
P ₄	364	3	

Given: page size = 1 kB

page table entry size = 4 bytes

Segment table entry size = 8 bytes

Maximum segment size = 256 kB.

Storage overhead for 2-level paging = P

Storage overhead for segmentation = S

Storage overhead for segmentation & paging = T.

Find the relation b/w these P, S and T.

For 2-level paging: (P).

given page size = 1 kB

page table entry size = 4 bytes

Assuming that size of page Table = page size = 1 kB

No. of entries in a page Table = 1024 bytes

$$= \frac{\text{Size of page table}}{\text{page table entry size}} = \frac{1024 \text{ bytes}}{4 \text{ bytes}} = 256$$

Storage overhead for process P₁ = 1 kB (for outer page table) +

1 kB (for inner page table) = 2 kB

" " " " P₂ = 1 kB (OPT) + 1 kB (IPT) = 2 kB

" " " " P₃ = 1 kB (OPT) + 1 kB (IPT) = 2 kB

" " " " P₄ = 1 kB (OPT) + 2 kB (IPT) = 3 kB

So, total overhead, P = (2 + 2 + 2 + 3) kB = 9 kB = 9216 bytes



for segmentation: (8)

segment table entry size = 8 bytes

maximum segment size = 256 KB

Storage overhead = Segment table entry size $\times$ No. of segments

storage overhead of process $P_1 = 8 \text{ bytes} \times 4 = 32 \text{ bytes}$

" " " " $P_2 = 8 \text{ bytes} \times 5 = 40 \text{ bytes}$

" " " " $P_3 = 8 \text{ bytes} \times 8 = 64 \text{ bytes}$

" " " " $P_4 = 8 \text{ bytes} \times 3 = 24 \text{ bytes}$

So, total overhead, $S = (32 + 40 + 64 + 24) \text{ bytes} = 160 \text{ bytes}$

for segmentation, with paging: (1)

Given,

page size = 1 KB

page table entry size = 4 bytes

segment table entry size = 8 bytes

Maximum segment size = 256 KB.

We have to apply segmentation & then paging.

calculate Page table size:- No. of entries in page table $\times$ page table entry size

$$= 256 \times 4$$

$$= 1024 \text{ bytes} = 1 \text{ KB}$$

Storage overhead = storage overhead of segmentation + 1 KB overhead for paging.

Storage overhead for process $P_1 = (8 \times 4) + 1 \text{ KB} = 1056 \text{ bytes}$

" " " " $P_2 = (8 \times 5) + 1 \text{ KB} = 1064 \text{ bytes}$

" " " " $P_3 = (8 \times 8) + 1 \text{ KB} = 1088 \text{ bytes}$

" " " " $P_4 = (8 \times 3) + 1 \text{ KB} = 1048 \text{ bytes}$

So, Total overhead, $T = (1056 + 1064 + 1088 + 1048) \text{ bytes} = 4256 \text{ bytes}$.

$\therefore P = 9 \text{ KB} = 9216 \text{ bytes}$

$S = 160 \text{ bytes}$

$T = 4256 \text{ bytes}$

$S < T < P$ $\leftarrow$ relation b/w P , S and T .